

AUDICLABS INDIA · FRESHER HIRING PROGRAMME 2026

Technical Assessment

Transcribe Hub: Multi-User Transcription Portal

Prepared for	**Aditya Kadepure**
Role	**Full Stack Developer**
Assessment ID	**AL-FS-02**

Issued October 2026 · API documentation: docs.audiclabs.com

CONFIDENTIAL - PREPARED FOR ADITYA KADEPURE - DO NOT SHARE

BEFORE YOU START

General Instructions

1 About This Assessment

This is a practical, take-home assessment based on a real problem in AudicLabs' speech AI platform. It is designed to show how you think, build and communicate. Every requirement in your assessment, including the Additional Requirements section, is mandatory.

2 Timeline and Submission

- **Deadline**: the submission date is shared by the AudicLabs hiring team in your assessment email.
- **Submit** by replying to that email with: your GitHub repository link (public, or private with access granted), your demo video link, and your write-up (PDF).
- **Follow-up review**: you will demo your solution and explain your design and code to the AudicLabs team.
- **Questions**: reply to the assessment email. Where something is unclear, make a reasonable assumption and write it down in your README.

3 Working Rules

- Open-source libraries and AI coding assistants may be used. Credit significant third-party code, and be ready to explain every part of your solution.
- The work must be your own. Do not share your assessment, API key or solution with other candidates.
- Use test or staging systems only. Never put credentials, API keys or confidential data into a repository or an LLM prompt.

4 AudicLabs API Ground Rules

- **API key**: AudicLabs will share a scoped API key with credits for this task. Read it from an environment variable or `.env` file, never commit it, and never put it in browser or mobile code.
- **Docs**: everything you need is at `https://docs.audiclabs.com` (APIs, Authentication, Errors, Rate limits). Read it before you start.
- **Errors and limits**: handle 401/403 (key or scope), 402 (credits exhausted), 413 (file too large), 422 (`invalid_audio`), 429 and 503 (respect `Retry-After`). Show the user a clear message, not a stack trace. Log the `X-Request-Id` of every call.
- **Credits are limited**: every transcription, TTS call or streaming session costs credits. Cache results while developing and do not call the API in loops you have not tested on one file first.

BEFORE YOU SUBMIT

Scoring and Submission Checklist

5 How Submissions Are Scored

Area	Weight	What we look for
Functionality	30%	Core requirements work end to end on real inputs; edge cases handled.
Technical depth	25%	Correct use of APIs and models, sound design, measured results.
Code quality	20%	Clear structure, naming, typing, tests, configuration and error handling.
Documentation and demo	15%	README, write-up and video explain what was built, why, and its limits.
Product thinking	10%	Usability, sensible defaults, and attention to the real end user.

Each assessment also lists its own evaluation criteria, which guide scoring within these areas. All requirements, including Additional Requirements, are mandatory and scored.

6 Submission Checklist

Item	Check before you send
Repository	Code runs from a clean clone. Dependencies pinned (requirements.txt, package-lock.json or similar). A `.env.example` lists every setting without real values.
README	Sections for overview, architecture, setup, configuration, how to run, how to test, assumptions and known limitations.
Secrets	No API keys, passwords or tokens anywhere in the code or git history.
Demo video	3 to 5 minutes. Show the main flow on real input, one error case, and where to find the results.
Write-up	1 to 2 pages (PDF): design decisions, measured results, what worked, limitations and next steps.
Sample outputs	Example inputs and the outputs your system produced, committed in a `samples/` folder.
Evidence	Any accuracy, latency or cost figures you report can be reproduced with a documented command.

REFERENCE

AudicLabs API Quick Reference

Base URL `https://api.audiclabs.com/v1/{service}/{endpoint}`. Authenticate with `Authorization: Bearer <key>` or `X-API-Key`. Full details: `https://docs.audiclabs.com`.

Service	Method and endpoint	Key details
Speech to Text	`POST /v1/stt/transcriptions`	Multipart file up to 100 MB; optional model, language (or auto-detect), prompt. Returns JSON text.
Speech to Text (live)	`WSS /v1/stt/stream`	start → ready → 16-bit mono PCM frames → partial / final → stop. 30 s idle timeout.
Text to Speech	`POST /v1/tts/speech`	JSON: model audic-tts-1, input, voice, speed, language. MP3, streamed by default.
Text to Speech (live)	`WSS /v1/tts/stream`	One utterance per connection: start → binary MP3 frames → done. Send cancel to stop.
Noise Suppressor	`POST /v1/noise-suppressor/denoise`	Multipart file; model audic-1 to audic-5. Returns cleaned WAV; stats in X-NS-Stats.
Speaker Diarization	`POST /v1/diarization/diarize`	Up to 200 MB / 4 hours. Returns speakers and segments with start / end times.
Browser tickets	`POST /v1/tickets`	Server-side only. Single-use, 60 seconds. Browser connects with ?ticket=.
Lookups (free)	`GET /v1/stt/languages` `GET /v1/tts/voices`	Also /stt/models, /tts/languages and /health for each service.

Limits and Conventions

Limit	Value
Rate limit	120 requests per minute (pay-as-you-go). Headers: RateLimit-Remaining, RateLimit-Reset, Retry-After on 429.
Credits	1 credit per transcription, TTS call, denoise call, diarization call or streaming session. 402 when exhausted.
Translation	Not part of the AudicLabs API. Use a translation model of your choice and justify it.
Tracing	Every response carries X-Request-Id. Log it with each call.

ASSESSMENT AL-FS-02 · FULL STACK DEVELOPER

Transcribe Hub: Multi-User Transcription Portal

ASSESSMENT ID	**AL-FS-02**	ROLE	Full Stack Developer
CANDIDATE	**Aditya Kadepure**	FOCUS AREA	Speech to Text
KEY SERVICES	STT (file), Speaker Diarization, Noise Suppressor, Languages / voices lookups		

1 Objective

Build a full stack web portal where teams upload lectures, interviews and podcasts, get searchable transcripts, and manage their work in shared projects. The product must handle many users, track how many AudicLabs credits each team uses, and keep the API key safely on the server.

2 AudicLabs APIs to Use

- **Speech to Text (file)**: `POST https://api.audiclabs.com/v1/stt/transcriptions`. Multipart upload of `file` (up to 100 MB), with optional `model` (`audic-stt-1`), `language` (e.g. `en-US`, `hi-IN`; omit for auto-detect) and `prompt` (names or terms to guide spelling). Returns JSON with `text`.
- **Speaker Diarization**: `POST https://api.audiclabs.com/v1/diarization/diarize`. Multipart `file` (up to 200 MB / 4 hours), optional `merge_gap`, `threshold`, `response_format`. Returns speakers and segments with `start` / `end` times.
- **Noise Suppressor**: `POST https://api.audiclabs.com/v1/noise-suppressor/denoise`. Multipart `file` plus optional `model` (`audic-1` to `audic-5`). Returns cleaned WAV; processing stats are in the `X-NS-Stats` header.
- **Lookups (free)**: `GET /v1/stt/languages`, `/v1/stt/models`, `/v1/tts/voices`, `/v1/tts/languages` and the `/health` endpoints. Load languages and voices from these at runtime; do not hardcode them.

3 Functional Requirements

A. Accounts and Workspaces

- Sign up, log in and log out with secure password hashing and JWT or session cookies.
- Workspaces with roles: **Owner** (manage members and quota), **Editor** (upload, edit), **Viewer** (read only). Enforce roles in the API, not just the UI.
- Invite members by email link; the owner can change roles or remove members.

B. Upload and Processing

- Upload audio or video (up to 100 MB) with progress, language selection (or auto), spelling hints, and options for noise suppression and speaker labels.
- Process in a background worker: optional `/v1/noise-suppressor/denoise`, optional `/v1/diarization/diarize`, then `/v1/stt/transcriptions`. Show live status (queued, processing, done, failed) in the UI without page refresh.
- Retry failed jobs; respect `Retry-After` on 429 and 503.

C. Transcript Experience

- Transcript page with audio player; speaker-labelled turns that can be renamed; inline editing with version history.
- Full-text search across all transcripts in a workspace, with highlighted matches.

- Export as TXT, DOCX and SRT; share a read-only public link that can be revoked.

D. Usage and Admin

- Record every AudicLabs call (service, credits, request ID, duration) per workspace.
- Owner dashboard: credits used per day and per member; monthly workspace quota that blocks new uploads when reached.

4 Recommended Technology

React or Next.js front end; Node.js (Express/NestJS) or Python (FastAPI/Django) back end; PostgreSQL; Redis-backed job queue; Docker Compose for the full stack. You may use other tools if you explain why.

5 Deliverables

1. Public or private GitHub repository (share access with AudicLabs) with a README covering setup, configuration and how to run.
2. A 3 to 5 minute screen recording of the working application.
3. A short write-up (1 to 2 pages): design decisions, what worked, known limitations, and what you would do next.

6 Evaluation Criteria

- Complete, working product flow from sign-up to exported transcript.
- Correct role-based access control enforced on the server.
- Reliable background processing and usage tracking.
- Code organisation across front end, back end and database (schema, migrations).

7 Additional Requirements

- Automated tests for the API covering auth, roles and the upload pipeline.
- Deployed demo URL (any free hosting) in addition to local Docker setup.

General instructions, the scoring rubric and the API ground rules at the front of this booklet apply to this assessment.